

กลุ่ม [1] — Introduction to Rust & Programming Paradigms

1. นางสาววิญญูตา การดี — Concept + Short Code Illustration
2. นายคิณภัสน์ แก้วพ่อง — Detailed Code + Live Demo
3. จักรภพ ภูมิพัฒน์ — Rust vs Other Language + PPL Analysis
4. นายณทรธรณ์ พรหมประดิษฐ์ — Exercises + Common Mistakes + Challenge

Agenda — โครงสร้างการนำเสนอ (รวม ~20 นาที)

แต่ละคนนำเสนอประมาณ 5 นาที ต่อเนื่องกันตามลำดับ

1

Member 1

**Concept + Short Code
Illustration**

แนวคิดหลักของหัวข้อ พร้อมโค้ดตัวอย่างสั้น ๆ

~5 นาที

2

Member 2

Detailed Code + Live Demo

อธิบายโค้ดเชิงลึกและสาธิตการทำงานจริง

~5 นาที

3

Member 3

**Rust vs Other Language +
PPL Analysis**

เปรียบเทียบภาษา และวิเคราะห์เชิง
Programming Language

~5 นาที

4

Member 4

**Exercises + Common
Mistakes + Challenge**

ข้อผิดพลาดที่พบบ่อย และโจทย์ท้าทายผู้ฟัง

~5 นาที

Introduced to Rust

Rust เป็นภาษาโปรแกรมระบบ โดยมีเป้าหมายหลักคือการมอบความเร็วและความปลอดภัยของหน่วยความจำโดยไม่ต้องพึ่งพา Garbage

Collector (GC) เหมือนภาษาอย่าง Python, Java

ความเป็นมาของ Rust

- ภาษา Rust เริ่มต้นในปี 2006 โดยมี Graydon Hoare วิศวกรซอฟต์แวร์ของ Mozilla เป็นผู้คิดค้น โดย Graydon พบว่าลิฟต์ที่อพาร์ทเมนต์เสียอีกครั้งเพราะซอฟต์แวร์ในระบบเกิด Crash จากปัญหาเรื่อง Memory Bug เขาจึงเริ่มพัฒนาภาษาใหม่ขึ้นมา โดยตั้งชื่อภาษาว่า Rust ตามชื่อของเชื้อรา Rust Fungi ที่ขึ้นชื่อเรื่องความอืด ถึก และทน
- ต่อมาในปี 2009 Mozilla เห็นศักยภาพของภาษา Rust ว่าสามารถนำมาแก้ปัญหาด้านความปลอดภัยและประสิทธิภาพของเว็บเบราว์เซอร์ได้ จึงได้สポンเซอร์โปรเจกต์นี้
- ปี 2012 Mozilla ประกาศพัฒนา Servo ซึ่งเป็น Browser Engine ยุคใหม่ด้วยภาษา Rust เพื่อพิสูจน์ว่า Rust สามารถประมวลผลแบบ Parallelism ได้อย่างปลอดภัย
- ปี 2014 ได้มีการคิดค้นระบบ Ownership & Borrowing รูปแบบปัจจุบันขึ้นมา เพื่อตัด Garbage Collector ออกไปโดยสิ้นเชิง
- ปี 2015 ได้ปล่อย Rust 1.0 ออกมาอย่างเป็นทางการ
- ปี 2020 - ปัจจุบัน เนื่องจากปัญหาทางการเงิน Mozilla ได้ปลดพนักงานทีม Rust ออกบางส่วน ทำให้บริษัทยักษ์ใหญ่

จุดเด่นของ Rust

1.Memory Safety

แก้ปัญหาเรื่อง Memory Leak หรือ Dangling Pointers ที่มักเกิดใน C/C++ ผ่านระบบ Ownership, Borrowing, และ Lifetimes โดยตัวตรวจสอบของคอมไพเลอร์ Borrow Checker จะตรวจสอบความถูกต้องของการใช้หน่วยความจำตั้งแต่ช่วง Compile

2.Fearless Concurrency

คือกฎการจัดการหน่วยความจำที่เข้มงวด ช่วยป้องกันปัญหา Data Race หรือก็คือการที่หลาย thread เข้าถึงหน่วยความจำตำแหน่งเดียวกันพร้อมกันโดยไม่มีการจัดระเบียบ ทำให้การเขียนโปรแกรมแบบทำงานพร้อมกันมีความปลอดภัยสูงมากขึ้น

3.Zero-Cost Abstractions

คือการใช้ฟีเจอร์ระดับสูง เช่น Iterators, Generics หรือ Closure ไม่ทำให้ประสิทธิภาพการทำงานลดลงเมื่อเทียบกับ การเขียนโค้ดระดับล่าง

4.Tooling แบบครบวงจร

5: มี Cargo ซึ่งเป็นทั้ง Package Manager และ Build System ในตัว ช่วยจัดการ Dependencies, การบิลด์โปรเจกต์,

Example basic code

Explain

- การทำงาน `fn main() { ... }` จุดเริ่มต้นการทำงานของโปรแกรม (Main function) โค้ดทั้งหมดจะเริ่มรันจากตรงนี้
- `let name = "Rust";` สร้างตัวแปรชื่อ `name` โดยค่าเริ่มต้นของ `Rust` ตัวแปรจะแก้ไขไม่ได้ (Immutable)
- `let mut score = 90;` สร้างตัวแปรชื่อ `score` โดยใส่คีย์เวิร์ด `mut` (Mutable) เพื่อบอกว่าตัวแปรนี้สามารถเปลี่ยนค่าได้
- `score = score + 10;` อัปเดตค่าใน `score` เพิ่มขึ้นอีก 10 (กลายเป็น 100)
- `println!(...);` คำสั่งพิมพ์ข้อความออกทางหน้าจอ โดยวงเล็บปีกกา `{}` จะถูกแทนที่ด้วยค่าของตัวแปรตามลำดับ (`name` และ `score`)

```
fn main() {
    let name = "Rust";
    let mut score = 90;
    score = score + 10;

    println!("Hello, {}! Your score is {}.", name, score);
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

(Output): PlaintextHello, Rust! Your score is 100.

Detailed Code + Live Demo

- Example 1

```
fn imperative(numbers: &[i32]) -> i32 {
    let mut total = 0;
    for &n in numbers {
        if n % 2 == 0 { total += n * n; }
    }
    total
}

fn functional(numbers: &[i32]) -> i32 {
    numbers.iter()
        .filter(|&&n| n % 2 == 0)
        .map(|&n| n * n)
        .sum()
}
```

```
struct NumberList {
    numbers: Vec<i32>,
}

impl NumberList {
    fn new(numbers: Vec<i32>) ->
    Self {
        NumberList { numbers }
    }

    fn sum_even_squares(&self) ->
    i32 {
        self.numbers
            .iter()
            .filter(|&&n| n % 2
            == 0)
            .map(|&n| n * n)
            .sum()
    }
}
```

Detailed Code + Live Demo

- Example 2

```
fn main() {  
    // ตัวแปรเป็น immutable โดย default  
    let x = 5;  
    println!("x = {}", x);  
  
    // ต้องประกาศ mut ถึงจะแก้ค่าได้  
    let mut y = 5;  
    y += 1;  
    println!("y = {}", y);  
  
    // ownership: การ "move" ค่า  
    let s1 = String::from("Rust");  
    let s2 = s1; // s1 ถูก move ไปให้ s2 แล้ว  
    println!("s2 = {}", s2);  
  
    // ถ้าต้องการใช้ทั้งสองตัว ต้อง clone  
    let s3 = s2.clone();  
    println!("s2 = {}, s3 = {}", s2, s3);  
}
```

Rust vs C++

Aspect	Rust	C++
Syntax	C-style, เน้น Expression-oriented	C-style, เน้น Statement-oriented
Semantics / Behavior	ตัวแปรเป็น Immutable โดยค่าเริ่มต้น (ต้องระบุ <code>mut</code> เพื่อให้แก้ไขได้)	ตัวแปรเป็น Mutable โดยค่าเริ่มต้น (ต้องระบุ <code>const</code> เพื่อไม่ให้แก้ไขได้)
Type System	Static & Strong, มี Type inference และไม่มี Implicit conversion ที่เสี่ยงต่อข้อผิดพลาด	Static & Strong, อนุญาตให้ทำ Implicit conversion ได้หลายกรณี (เช่น <code>int</code> เป็น <code>float</code>)
Memory Management	จัดการผ่านระบบ Ownership และ Borrow Checker ในตอน Compile-time (ไม่มี GC)	จัดการด้วยตนเอง (<code>new/delete</code>) หรือใช้ Smart Pointers (RAII) เพื่อช่วยจัดการ
Safety	การันตี Memory Safety และ Thread Safety ตั้งแต่ตอน Compile จะไม่เกิด Segfaults ถ้าไม่ใช้ <code>unsafe block</code>	ไม่มีระบบป้องกันที่เข้มงวด โปรแกรมเมอร์ต้องรับผิดชอบเรื่อง Memory Leak, Dangling Pointers และ Data Races เอง

Analysis: C++ เน้นสถาปัตยกรรมแบบ Inheritance (สืบทอด) และให้อิสระสูงสุดในการจัดการหน่วยความจำ ในขณะที่ Rust เน้น Composition (การประกอบร่าง) แยกข้อมูลกับพฤติกรรมออกจากกัน และเลือกหลักการด้านความปลอดภัยไปให้ Compiler ตรวจสอบอย่างเข้มงวด (Ownership) เพื่อการันตีความปลอดภัยโดยไม่ลดประสิทธิภาพ (Zero-cost Abstraction)

PPL Perspective — มุมมองเชิง Programming Language

Syntax

Rust ใช้ C-style syntax เพื่อความคุ้นเคย แต่ปรับโครงสร้างเป็น **Expression-oriented** (คำสั่งส่วนใหญ่คืนค่าได้) ทำให้ลดความซับซ้อนของการประกาศตัวแปรชั่วคราว

Semantics

เน้นพฤติกรรม **Immutability by default** (แก้ค่าไม่ได้ถ้าไม่ระบุ mut) และใช้ **Move Semantics** แทนการคัดลอกข้อมูล เพื่อป้องกันผลกระทบข้างเคียง

Type System

เป็น **Statically** และ **Strongly Typed** ตรวจสอบตั้งแต่ตอนคอมไพล์ พร้อมฟีเจอร์ **Type Inference** ช่วยอนุมานชนิดข้อมูลอัตโนมัติ ลดความยาวของโค้ด

Memory / Resource

ไม่ต้องจัดการเมมโมรีเอง และไม่มี GC แต่ใช้กลไก **Ownership, Borrowing** และ **Lifetimes** เช็คความถูกต้องระดับคอมไพเลอร์ แก้ปัญหา Memory Leak

Abstraction

เป็นภาษา **Multi-paradigm** ดึงข้อดีจาก Imperative, Functional และปรับปรุง OOP โดยใช้ **Traits** แทนการสืบทอด (Composition over Inheritance)

Why Rust?

Rust ออกแบบมาเพื่อทำลายข้อจำกัด "ความเร็ว vs ความปลอดภัย" โดยนำเสนอ **Fearless Concurrency** ที่ทำงานเร็วเทียบเท่า C/C++ แต่ปลอดภัยต่อข้อมูลสูงสุด

Example 1 — Functional Programming

เขียนโปรแกรมที่รับคอลเลกชันของตัวเลข [1, 2, 3, 4, 5] แล้วเพิ่มค่าของตัวเลขทุกตัวเป็น 2 เท่า และแสดงผลลัพธ์ออกมา

Explain

โปรแกรมนี้สร้าง **Vector** ที่เก็บตัวเลขตั้งแต่ 1 ถึง 5 จากนั้นใช้ `.iter()` เพื่อวนดูข้อมูลแต่ละตัว และใช้ `.map()` เพื่อคูณค่าของแต่ละตัวด้วย 2 หลังจากนั้น `.collect()` จะนำค่าที่ได้มารวมเป็น **Vector** ตัวใหม่ชื่อ `result`

```
fn main() {  
    let numbers = vec![1, 2, 3, 4, 5];  
  
    let result: Vec<i32> = numbers  
        .iter()  
        .map(|x| x * 2)  
        .collect();  
  
    println!("{:?}", result);  
}
```

Example 2 — Functional Programming

เขียนโปรแกรมสร้างข้อมูลนักเรียน
ที่มี name และ score แล้วสร้าง
method สำหรับแสดงข้อมูลของ
นักเรียน

Explain

ใช้ `struct` สำหรับเก็บข้อมูลนักเรียน และใช้ `impl` เพื่อสร้าง
method `show_info()` สำหรับแสดงชื่อกับคะแนน โดยเป็น
ตัวอย่างแนวคิด OOP ใน Rust.

```
struct Student {  
    name: String,  
    score: i32,  
}  
  
impl Student {  
    fn show_info(&self) {  
        println!("{}", self.name, self.score);  
    }  
}  
  
fn main() {  
    let student = Student {  
        name: String::from("Bob"),  
        score: 85,  
    };  
  
    student.show_info();  
}
```

Common Mistakes 1 — ข้อผิดพลาดที่พบบ่อย

✗ Incorrect

```
fn main() {  
    let name = String::from("Rust");  
    let copy = name;  
  
    println!("{}", name);  
}
```

✓ Correct

```
fn main() {  
    let name = String::from("Rust");  
    let copy = name.clone();  
  
    println!("{}", name);  
}
```

ทำไมถึงผิด? (Why?)

เพราะเมื่อเขียน `let copy = name;` ค่า `String` จะถูก **Move Ownership** จาก `name` ไปให้ `copy` ทำให้ `name` ใช้งานต่อไม่ได้ โดยหลักการ Rust ที่เกี่ยวข้องก็จะมี **Ownership** และ **Move**

Common Mistakes 2 — ข้อผิดพลาดที่พบบ่อย

✗ Incorrect

```
fn main() {
    let mut text = String::from("Hello");

    let a = &mut text;
    let b = &mut text;

    a.push_str(" Rust");
    b.push_str("!");
}
```

ทำไมถึงผิด? (Why?)

Rust ไม่อนุญาตให้มี **Mutable Reference (&mut)** หลายตัวที่ใช้งานพร้อมกันกับข้อมูลเดียวกัน เพราะอาจทำให้เกิดการแก้ไขข้อมูลชนกัน

✓ Correct

```
fn main() {
    let mut text = String::from("Hello");

    {
        let a = &mut text;
        a.push_str(" Rust");
    }

    let b = &mut text;
    b.push_str("!");

    println!("{}", text);
}
```

Mutable Borrowing, Borrowing Rules, Memory Safety

Challenge — คำถามท้าทายผู้ฟัง

โจทย์ (Problem)

```
fn main() {  
    let score = 10;  
  
    score = score + 5;  
  
    println!("{}", score);  
}
```



Hint

โดยค่าเริ่มต้น ตัวแปรใน Rust จะเปลี่ยนค่าไม่ได้

```
//Solution  
fn main() {  
    let mut score = 10;  
  
    score = score + 5;  
  
    println!("{}", score);  
}
```

Key Takeaways — สรุปสิ่งที่ควรจำ

1

Rust เป็นภาษาที่เน้น Performance และ Memory Safety

ทำงานได้รวดเร็วใกล้เคียง C/C++ และช่วยลดข้อผิดพลาดด้านหน่วยความจำตั้งแต่ตอน Compile

2

Rust เป็นภาษาแบบ Multi-Paradigm

รองรับทั้ง **Imperative, OOP และ Functional Programming** ทำให้เลือกแนวทางเขียนโปรแกรมได้ตามลักษณะงาน

3

แนวคิดสำคัญของ Rust คือ Ownership, Borrowing และ Mutability

แนวคิดเหล่านี้ช่วยควบคุมการใช้ข้อมูลและหน่วยความจำให้ปลอดภัย โดยไม่จำเป็นต้องใช้ Garbage Collector

4

Rust แสดงให้เห็นความสัมพันธ์ระหว่างการออกแบบภาษากับแนวคิด PPL

เช่น Type System, Memory Management, Programming Paradigms และการตรวจสอบข้อผิดพลาดที่ Compile Time

References — เอกสารอ้างอิง

- The Rust Programming Language (The Rust Book) — <https://doc.rust-lang.org/book/>
- Rust by Example — <https://doc.rust-lang.org/rust-by-example/>
- C++ Core Guidelines — <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines>
- Rust tutorial — https://www.w3schools.com/rust/rust_intro.php

GitHub Repository: [ลิงก์ repository ของกลุ่ม]

Questions & Answers

กลุ่ม [1] — **[Introduction to Rust & Programming Paradigms]**